



Lec 22

Python Functions, Types of Functions Arguments

What are Functions?

python function is a block of code that performs a specific task and can be reused multiple times throughout a program

block -

if <condition> :

block 1

else :

block 2

Advantages of Function

1. Code reusability :

We can reuse the code multiple times.

Eg :

Normal way :

a = 5

b = 10

c = a + b

print(c)

a = 5

b = 10

c = a + b

print(c)

← addition

← addition

Using function :

def add() :

a = 5

b = 10

c = a + b

print(c)

← add

(just call function)

add()

← function call



2. Modularity:

We convert complex program into smaller manageable code. every function performs a specific task.

3. Ease of ~~main~~ maintenance:

- Easy to modify & update function
- Due to multiple functions we can recall particular function.

4. Testing & debugging:

Easy for testing & debugging

5. Improved Readability:

We can provide meaningful names to the function so that it is easy to understand

~ These are 2 types of Functions:

1. Built in Function - readily available

Eg: 1. len()

2. min()

3. max()

4. any()

5. all()

6. Sorted()

7. reversed()

8. id()

9. type()

10. Sum()

11. print(), input()

12. str(), list(), tuple, set(), dict()

2. User-defined Functions

We use `def` keyword to define a function, then we write `<function-name>` & then provide parentheses & color. This is called function definition.

Syntax : `def <func-name>():`

Eg:-

```
def add():  
    a = 5  
    b = 10  
    c = a + b  
    print(c)
```

Annotations:

- keyword (points to `def`)
- function name (points to `add`)
- block (points to the indented code lines)
- addition logic (points to `c = a + b`)

To call the function we write function name & parentheses

Fe Eg. 1

def add():

function def

add()

function call

~ Convention for function names

1. user lowercase
2. join the multiple words with underscore (-)
3. If two functions name is same the latest function logic will be executed.

Eg. `def add():`

```
def add():
```

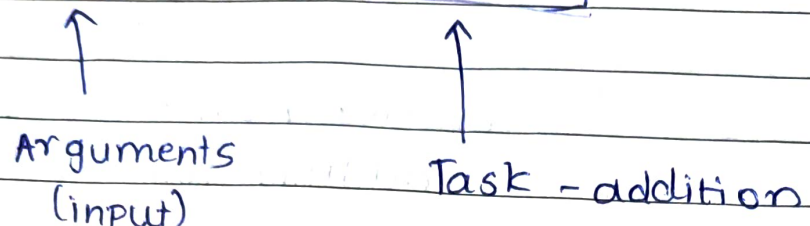
`def add():` ← latest func

 Logic will
 executed

def add1() :

II ← diff func
name

2



- 1) We pass some arguments to a function - input
- 2) function returns some value - output

In this way we communicate with function.

Eg. `def calculate_Sum(a,b):`
`sum = a + b`
`return sum`] fun def

Parameters

result = calculate_sum(10, 20)] fun call

↑ Arguments

2

- If we don't use return in the function definition, it returns None by default.



Types of Arguments :

1. Positional Arguments -

- This is the most common type.
- We pass the value on the basis of positions & assign it to the parameters & we make sure passing values in the correct position.

Eg.

```
def greet(name, age):  
    print("my name is", name, "and age is",  
          age)  
greet("bhakti", 21)  
> my name is bhakti and age is 21
```

```
def greet(name, age):  
    print("my name is", name, "and age is",  
          age)  
greet(21, "bhakti")  
> my name is 21 and age is bhakti
```

2. Keyword Argument -

- We can pass Arguments in any order
- We match the keywords with the parameters used in the function.

Eg.

```
def greet(name, age):  
    print("my name is : ", name, "and age is", age)  
greet(name="rahu", age=20)
```

3. Default Arguments -

- We can provide default values to a parameter & if we do not pass a value then it will consider this default value.

Eg. **Without default Argument**

```
def greet(name, age):  
    print("my name is : ", name, "and  
    age is : ", age)  
greet("rahul")
```

> error_TypeError

With default Argument

```
def greet(name="bhakti", age=21):  
    print("my name is ", name, "and age  
    is ", age)  
greet("bhakti priya")
```

> My name is priya and age is 21

```
greet("priya", 25)
```

> my name is priya and age is 25

```
greet(30)
```

> my name is bhakti and age is 30